

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ — PUCPR
ESCOLA POLITÉCNICA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO



LEONARDO DOS SANTOS MARQUES

WASHCONTROL

Sistema SaaS de Gestão para Lava-Cars e Estéticas Automotivas
Análise e Projeto de Sistema Computacional

CURITIBA
2026

LEONARDO DOS SANTOS MARQUES

WASHCONTROL

Sistema SaaS de Gestão para Lava-Cars e Estéticas Automotivas
Análise e Projeto de Sistema Computacional

Projeto de análise e especificação de sistema computacional elaborado como artefato avaliativo da disciplina de **Modelagem de Sistemas Computacionais** do Curso de Bacharelado em Ciência da Computação da PUCPR.

Professora: Patricia Rucker de Bassi

Disciplina: Modelagem de Sistemas Computacionais

Semestre: 2026-1

CURITIBA

2026

SUMÁRIO

1	INTRODUÇÃO	5
1.1	Contexto de Negócio	5
1.2	Justificativa do Projeto	5
1.3	Benefícios	5
2	REQUISITOS	6
2.1	Requisitos Funcionais	6
2.2	Requisitos Não Funcionais (FURPS+)	7
3	RA1 — MODELO DE DOMÍNIO E CASOS DE USO	8
3.1	Diagrama de Caso de Uso	8
3.2	Especificações de Casos de Uso	9
3.2.1	UC-01 — Criar Ordem de Serviço	9
3.2.2	UC-03 — Atualizar Status da OS	11
3.2.3	UC-11 — Criar Agendamento	13
4	RA2 — DIAGRAMAS ESTRUTURAIS	15
4.1	Modelo de Domínio	15
4.2	Diagrama de Classes	16
4.3	Diagrama de Objetos	18
5	RA3 — DIAGRAMAS COMPORTAMENTAIS	19
5.1	Diagramas de Sequência	19
5.2	Diagramas de Atividades	24
5.3	Diagramas de Máquina de Estados	28
6	PROTÓTIPO E NAVEGAÇÃO DE TELAS	30
7	CRIATIVIDADE	32
8	RESPONSABILIDADES	33

1 INTRODUÇÃO

1.1 Contexto de Negócio

O mercado de cuidados automotivos — popularmente chamado de *detailing* — tem crescido expressivamente no Brasil. As pessoas investem cada vez mais em seus veículos e buscam serviços de qualidade. No entanto, a maioria dos estabelecimentos de pequeno e médio porte ainda opera de forma desorganizada: filas sem controle, anotações em papel, comunicação dispersa via aplicativos de mensagem e ausência de visibilidade financeira.

O **WashControl** é um sistema *Software as a Service* (SaaS) multi-tenant desenvolvido para atender lava-cars e estéticas automotivas. Cada estabelecimento cadastrado na plataforma é tratado como um *tenant* isolado, com seus próprios clientes, veículos, funcionários, ordens de serviço e dados financeiros. A plataforma é administrada centralmente por um Super Admin, que gerencia os tenants, planos de assinatura e faturamento.

A arquitetura multi-tenant permite que o WashControl atenda simultaneamente dezenas de estabelecimentos com um único sistema, reduzindo custos de infraestrutura e facilitando a escalabilidade do produto.

1.2 Justificativa do Projeto

O principal problema que o WashControl resolve é a **falta de previsibilidade e controle** nos lava-cars. Quando um cliente deixa o veículo e recebe a informação "volta em 2 horas", mas ao retornar o serviço nem foi iniciado, a confiança é destruída. Da mesma forma, o dono do estabelecimento frequentemente não sabe com precisão quanto lucrou no dia, quais funcionários são mais produtivos ou quais insumos estão em falta.

O projeto se justifica pela necessidade de automatizar o fluxo operacional — desde o agendamento até a entrega do veículo — e fornecer visibilidade financeira em tempo real ao gestor. Estabelecimentos que utilizam sistemas organizados passam mais credibilidade ao cliente, conseguem atender maior volume de serviços sem perda de qualidade e tomam decisões baseadas em dados concretos.

1.3 Benefícios

Para o dono do negócio (Manager):

- Controle financeiro real com Demonstrativo de Resultado do Exercício (DRE) automático;
- Visibilidade da produtividade da equipe com cálculo automático de comissões;
- Gestão de estoque integrada ao fluxo de atendimento;
- Programa de fidelidade configurável para reter clientes.

Para o lavador (Washer):

- Interface simplificada e responsiva para uso no pátio via celular;
- Fila de serviços organizada, eliminando conflitos de atribuição.

Para a operação como um todo:

- Redução de erros humanos (dupla marcação, serviços não cobrados);
 - Rastreabilidade completa do ciclo de vida de cada ordem de serviço;
 - Isolamento total de dados entre estabelecimentos (multi-tenancy seguro).
-

2 REQUISITOS

2.1 Requisitos Funcionais

Quadro 1 — Requisitos Funcionais do WashControl

ID	Descrição	Requisitante	Detalhes
RF01	O sistema deve permitir o cadastro e gerenciamento de Tenants (lava-jatos parceiros).	SUPER_ADMIN	Criação da conta principal da loja no modelo SaaS, com nome, slug único e plano de assinatura.
RF02	O sistema deve permitir o cadastro de clientes e seus respectivos veículos.	MANAGER	Suporte à vinculação de múltiplos veículos a um único cliente. Placa única por tenant.
RF03	O sistema deve gerenciar o ciclo de vida da Ordem de Serviço (OS) através de um Kanban.	MANAGER	A OS transita pelos status: PENDING, WAITING_QUEUE, IN_PROGRESS, READY, COMPLETED ou CANCELED.
RF04	O sistema deve permitir a visualização e atualização de status das OSs ativas no pátio.	WASHER	Interface simplificada para que o lavador mova o card da OS (ex: WAITING_QUEUE → IN_PROGRESS).
RF05	O sistema deve permitir o agendamento prévio (Appointment) de serviços de lavagem.	CLIENTE MANAGER	O cliente ou o gerente podem reservar horário. Um agendamento gera no máximo uma OS (idempotência).
RF06	O sistema deve abater automaticamente produtos do estoque ao concluir uma OS.	MANAGER	Integração entre a conclusão da OS (COMPLETED) e a baixa automática

ID	Descrição	Requisitante	Detalhes
			dos insumos utilizados.
RF07	O sistema deve gerar uma transação financeira de receita ao concluir uma OS.	MANAGER	O fechamento da OS no Kanban engatilha o registro financeiro automático com método de pagamento.
RF08	O sistema deve permitir o registro e controle de despesas fixas e pagamentos de funcionários.	MANAGER	Controle de custos operacionais como água, luz, aluguel e comissões calculadas por OS concluída.
RF09	O sistema deve disparar notificações sobre a mudança de status do veículo.	CLIENTE	O cliente deve ser avisado (ex: via WhatsApp ou Push) quando o status mudar para READY.
RF10	O sistema deve prover relatórios gerenciais sobre volume de OSs, receitas e despesas.	MANAGER	Dashboard unificando dados financeiros e operacionais do Tenant (DRE, fluxo de caixa, KPIs).

Fonte: Elaborado pelo autor (2026).

2.2 Requisitos Não Funcionais (Classificação FURPS+)

Quadro 2 — Requisitos Não Funcionais do WashControl (FURPS+)

ID	Descrição	Requisitante	Categoria FURPS+	Detalhes
RNF01	O sistema deve possuir interface responsiva para smartphones.	WASHER	Usability	Essencial para uso no pátio via celular, com mãos potencialmente molhadas.
RNF02	O sistema deve garantir isolamento lógico dos dados entre diferentes lojas.	SUPER_ADMIN	Reliability	Um gerente nunca pode acessar dados de outro tenant. O tenantId é obrigatório em todas as queries.
RNF03	A atualização de status da OS deve ser refletida na interface em no máximo 2 segundos.	MANAGER	Performance	Tempo de resposta que garante fluidez da operação no pátio em tempo real.
RNF04	O sistema deve utilizar o padrão Repository com ORM (Prisma) para acesso a dados.	Dev. Team	Supportability	Facilita manutenção do código, trocas de SGBD e escalabilidade do sistema.
RNF05	O acesso a funcionalidades financeiras e de estoque deve ser restrito ao papel MANAGER.	MANAGER	+ Security	O WASHER possui apenas acesso operacional (pátio, OS), sem visibilidade financeira.
RNF06	O modelo de dados central do sistema	Arquitetura	+ Implementation	Restrição tecnológica

ID	Descrição	Requisitante	Categoria FURPS+	Detalhes
	deve ser armazenado em banco de dados relacional.			atendida pelo uso do schema.prisma com PostgreSQL.

Fonte: Elaborado pelo autor (2026).

3 RA1 — MODELO DE DOMÍNIO E CASOS DE USO

3.1 Diagrama de Caso de Uso

O diagrama de caso de uso a seguir apresenta os quatro atores do sistema — **Super Admin**, **Manager**, **Washer** e **Cliente** — e seus respectivos casos de uso distribuídos em cinco pacotes funcionais. As relações de <<include>> e <<extend>> modelam as automações disparadas pelo sistema ao concluir uma OS.

Figura 1 — Diagrama de Caso de Uso do WashControl

```

@startuml CasosDeUso_WashControl
left to right direction
skinparam packageStyle rectangle

actor "Manager\n(Dono/Gerente)" as MGR
actor "Washer\n(Lavador)" as WSH
actor "Cliente" as CLI
actor "Super Admin" as SA
actor "Sistema" as SYS <<system>>

rectangle "WashControl SaaS" {
    package "Gestão de OS" {
        usecase "Criar OS" as UC01
        usecase "Adicionar Itens à OS" as UC02
        usecase "Atualizar Status da OS" as UC03
        usecase "Cancelar OS" as UC04
        usecase "Visualizar Board Kanban" as UC05
        usecase "Registrar Pagamento" as UC06
    }
    package "Clientes e Veículos" {
        usecase "Cadastrar Cliente" as UC07
        usecase "Cadastrar Veículo" as UC08
        usecase "Consultar Histórico" as UC09
        usecase "Aplicar Desconto Fidelidade" as UC10
    }
    package "Agendamentos" {
        usecase "Criar Agendamento" as UC11
        usecase "Converter em OS" as UC12
        usecase "Cancelar Agendamento" as UC13
    }
    package "Financeiro e Estoque" {
        usecase "Visualizar DRE" as UC14
        usecase "Lançar Despesa Manual" as UC15
        usecase "Gerenciar Insumos" as UC18
        usecase "Baixar Estoque por OS" as UC19
        usecase "Calcular Comissão" as UC17
    }
    package "Administração SaaS" {
        usecase "Gerenciar Tenants" as UC24
        usecase "Configurar Plano SaaS" as UC25
        usecase "Ativar/Desativar Tenant" as UC26
    }
}

MGR --> UC01 ; MGR --> UC02 ; MGR --> UC03 ; MGR --> UC04
MGR --> UC05 ; MGR --> UC06 ; MGR --> UC07 ; MGR --> UC08
MGR --> UC09 ; MGR --> UC11 ; MGR --> UC12 ; MGR --> UC14
MGR --> UC15 ; MGR --> UC18

WSH --> UC03 ; WSH --> UC05
CLI --> UC11
SA --> UC24 ; SA --> UC25 ; SA --> UC26

UC12 ..> UC01 : <<include>>
UC03 ..> UC19 : <<extend>> [ao COMPLETED]
UC03 ..> UC17 : <<extend>> [ao COMPLETED]
SYS --> UC10 ; SYS --> UC19 ; SYS --> UC17
@enduml

```

Fonte: Elaborado pelo autor (2026). Renderizar em planttext.com para exportar como imagem.

3.2 Especificações de Casos de Uso

As especificações a seguir detalham os três casos de uso mais representativos do WashControl, cobrindo o fluxo de criação de uma Ordem de Serviço (UC-01), a atualização de seu status ao longo do ciclo de vida (UC-03) e o processo de agendamento prévio de atendimento (UC-11).

3.2.1 UC-01 — Criar Ordem de Serviço

UC-01 — Criar Ordem de Serviço	
Breve Descrição	Permite que o Manager registre a entrada de um veículo no sistema, criando uma Ordem de Serviço vinculada a um cliente, um veículo e contendo os itens de serviço e/ou insumos a serem executados. A OS é criada com status PENDING e passa a compor o board Kanban.
Ator Principal	Manager
Ator Secundário	Sistema — valida dados, persiste registros, verifica fidelidade.
Pré-condições	• O Manager deve estar autenticado com papel MANAGER; • O cliente deve estar cadastrado no sistema (ou ser cadastrado durante o fluxo); • O veículo deve estar vinculado ao cliente; • Deve haver pelo menos um produto/serviço no catálogo do tenant.
Pós-condições	• Uma nova OS é criada com status PENDING no banco de dados; • A OS aparece na coluna PENDING do board Kanban; • Os itens (OrderItems) estão vinculados à OS com preços e quantidades.
Fluxo de Eventos	<u>Fluxo Básico</u> 1. O Manager acessa o módulo de Ordens de Serviço. 2. O Manager clica em "Nova OS". 3. O sistema exibe o formulário com campos: cliente, veículo, itens, observações e pagamento antecipado. 4. O Manager seleciona o cliente pelo nome ou telefone. 5. O sistema lista os veículos vinculados ao cliente selecionado. 6. O Manager seleciona o veículo a ser atendido. 7. O Manager adiciona os itens da OS (serviços e/ou insumos) com quantidades e preços unitários. 8. O sistema calcula e exibe o valor total em tempo real.

9. O Manager confirma a criação clicando em "Criar OS".

10. O sistema persiste a OS com status PENDING e redireciona para o board Kanban.

11. O sistema verifica se o cliente atingiu o intervalo de fidelidade e, se aplicável, sinaliza desconto.

Fluxo Alternativo — Cliente não cadastrado

4a. O Manager não encontra o cliente na busca.

4b. O Manager clica em "Novo Cliente".

4c. O sistema exibe o formulário de cadastro de cliente (nome, telefone, e-mail).

4d. O Manager preenche, confirma o cadastro e o fluxo retorna ao passo 5.

Fluxo Alternativo — Criação via Agendamento

1a. O Manager converte um agendamento existente em OS (UC-12).

1b. O sistema preenche automaticamente cliente e veículo a partir do agendamento.

1c. O fluxo continua a partir do passo 7.

Restrição: o sistema bloqueia a criação de segunda OS para o mesmo agendamento (idempotência via Appointment.orderId @unique).

Fluxo de Exceção — OS sem itens

9a. O Manager tenta confirmar a OS sem nenhum item adicionado.

9b. O sistema exibe: *"A OS deve conter pelo menos um serviço ou insumo"*.

9c. O fluxo retorna ao passo 7.

3.2.2 UC-03 — Atualizar Status da Ordem de Serviço

UC-03 — Atualizar Status da Ordem de Serviço	
Breve Descrição	Permite que o Manager ou o Washer movam a OS entre os estados do ciclo de vida (PENDING → WAITING_QUEUE → IN_PROGRESS → READY → COMPLETED). Ao atingir COMPLETED, o sistema executa automaticamente os processos financeiro, de estoque e de fidelidade.
Ator Principal	Manager (todos os status) Washer (IN_PROGRESS e READY)
Ator Secundário	Sistema — registros financeiros e de estoque automáticos.
Pré-condições	• O usuário deve estar autenticado; • A OS deve existir e pertencer ao tenant do usuário; • A transição de status deve ser válida conforme o ciclo de vida definido.
Pós-condições	• O status da OS é atualizado no banco de dados; • Os timestamps correspondentes são registrados (startedAt, finishedAt, completedAt); • Se COMPLETED: FinancialTransaction, InventoryTransaction, totalWashes++ e comissão são gerados automaticamente; • A interface é atualizada em no máximo 2 segundos (RNF03).
Fluxo de Eventos	<u>Fluxo Básico — Conclusão de OS (READY → COMPLETED)</u> 1. O Manager localiza a OS na coluna READY do board Kanban. 2. O Manager clica em "Concluir / Registrar Pagamento". 3. O sistema exibe o formulário de conclusão: método de pagamento, valor total e valor adiantado. 4. O Manager seleciona o método de pagamento (PIX, Dinheiro, Cartão Crédito ou Débito). 5. O Manager clica em "Confirmar Conclusão".

6. O sistema atualiza o status para COMPLETED e registra completedAt = now().
7. O sistema cria automaticamente: FinancialTransaction (INCOME), InventoryTransaction OUT por insumo utilizado e decrementa Product.stock.
8. O sistema incrementa Customer.totalWashes e verifica o programa de fidelidade.
9. O sistema calcula e registra a comissão do Employee vinculado.
10. O card da OS é movido para o histórico de OSs concluídas.

Fluxo Alternativo — Lavador inicia serviço (WAITING_QUEUE → IN_PROGRESS)

1. O Washer acessa a visão do pátio.
2. O Washer clica em "Iniciar" no card do veículo.
3. O sistema atualiza o status para IN_PROGRESS e registra startedAt = now().
4. O card é movido para IN_PROGRESS no Kanban do Manager.

Fluxo Alternativo — Cancelamento

1. O Manager seleciona "Cancelar OS" no menu do card.
2. O sistema solicita confirmação.
3. O sistema atualiza o status para CANCELED.

Restrição: somente MANAGER pode cancelar uma OS (RNF05).

Fluxo de Exceção — Transição inválida

1. O sistema rejeita transições fora do ciclo definido (ex: PENDING → COMPLETED).
2. O sistema exibe mensagem informando os status válidos para transição.

3.2.3 UC-11 — Criar Agendamento

UC-11 — Criar Agendamento	
Breve Descrição	Permite que o Manager registre previamente um horário de atendimento para um cliente e veículo, reservando uma vaga na agenda do lava-car. O agendamento permanece com status SCHEDULED até ser convertido em OS ou cancelado.
Ator Principal	Manager
Ator Secundário	Sistema — valida disponibilidade de horário e notifica o cliente.
Pré-condições	• O Manager deve estar autenticado com papel MANAGER; • O cliente e veículo devem estar cadastrados no sistema; • O horário solicitado não deve estar ocupado por outro agendamento.
Pós-condições	• Um novo Appointment é criado com status SCHEDULED; • O agendamento aparece no calendário do sistema; • O cliente pode receber uma notificação de confirmação (RF09).
Fluxo de Eventos	<u>Fluxo Básico</u> 1. O Manager acessa o módulo de Agendamentos. 2. O Manager clica em "Novo Agendamento". 3. O sistema exibe o formulário: data/hora, cliente, veículo, serviço desejado e observações. 4. O Manager seleciona a data e horário desejados. 5. O sistema verifica a disponibilidade do horário selecionado. 6. O Manager seleciona o cliente e o veículo. 7. O Manager confirma o agendamento. 8. O sistema persiste o Appointment com status SCHEDULED e exibe no calendário. <u>Fluxo Alternativo — Conversão em OS</u>

1. No dia do atendimento, o Manager clica em "Converter em OS".
2. O sistema cria uma OS pré-preenchida com os dados do agendamento.
3. O sistema vincula a OS ao agendamento via Appointment.orderId.
4. O status do agendamento é atualizado para COMPLETED.

Fluxo de Exceção — Conflito de horário

- 5a. O sistema detecta outro agendamento no mesmo horário.
- 5b. O sistema exibe: *"Horário indisponível. Selecione outro horário."*
- 5c. O Manager seleciona horário alternativo e o fluxo retorna ao passo 6.

Fluxo de Exceção — Idempotência (OS já existente)

- 1a. O Manager tenta converter um agendamento que já possui OS vinculada.
- 1b. O sistema rejeita a operação com a mensagem: *"Este agendamento já possui uma OS vinculada"*.
- 1c. O sistema redireciona para a OS existente (Appointment.orderId @unique).

4 RA2 — DIAGRAMAS ESTRUTURAIS

4.1 Modelo de Domínio

O modelo de domínio apresenta as entidades do problema de negócio, seus atributos essenciais, multiplicidades e relacionamentos conceituais. Nesta representação não há foco em aspectos técnicos de implementação, mas sim no vocabulário do domínio do lava-car.

Figura 2 — Modelo de Domínio do WashControl

```

@startuml ModeloDominio_WashControl
skinparam classAttributeIconSize 0

class "Lava-Car (Tenant)" {
    nome; slug; plano; meta mensal
    intervalo de fidelidade; desconto fidelidade
}
class "Usuário" { nome; e-mail; papel }
class "Cliente" { nome; telefone; total de lavagens }
class "Veículo" { placa; marca; modelo; cor; ano; tipo }
class "Ordem de Serviço" {
    status; valor total; pagamento
    data início; data conclusão
}
class "Item da OS" { nome; quantidade; preço unitário; é serviço? }
class "Produto/Serviço" { nome; categoria; preço; estoque; mínimo }
class "Agendamento" { data/hora; status; observações }
class "Funcionário" { nome; cargo; salário; comissão% }
class "Transação Financeira" { tipo; descrição; valor; status }
class "Movimentação de Estoque" { tipo; quantidade }
class "Despesa Fixa" { nome; valor; dia vencimento }

"Lava-Car (Tenant)" "1" -- "0..*" "Usuário"
"Lava-Car (Tenant)" "1" -- "0..*" "Cliente"
"Lava-Car (Tenant)" "1" -- "0..*" "Veículo"
"Lava-Car (Tenant)" "1" -- "0..*" "Ordem de Serviço"
"Lava-Car (Tenant)" "1" -- "0..*" "Produto/Serviço"
"Lava-Car (Tenant)" "1" -- "0..*" "Funcionário"
"Lava-Car (Tenant)" "1" -- "0..*" "Despesa Fixa"
"Cliente" "1" -- "1..*" "Veículo"
"Cliente" "1" -- "0..*" "Ordem de Serviço"
"Cliente" "1" -- "0..*" "Agendamento"
"Veículo" "1" -- "0..*" "Ordem de Serviço"
"Ordem de Serviço" "1" *-- "1..*" "Item da OS"
"Ordem de Serviço" "1" -- "0..1" "Agendamento"
"Item da OS" "0..*" -- "0..1" "Produto/Serviço"
"Produto/Serviço" "1" -- "0..*" "Movimentação de Estoque"
@enduml

```

Fonte: Elaborado pelo autor (2026).

4.2 Diagrama de Classes

O diagrama de classes a seguir foi construído em correspondência direta com o `schema.prisma` do WashControl, mapeando todas as entidades, atributos, operações e relacionamentos (composição, associação e dependência) em notação UML.

Figura 3 — Diagrama de Classes do WashControl

```

@startuml DiagramaClasses_WashControl
skinparam classAttributeIconSize 0

enum UserRole { SUPER_ADMIN; MANAGER; WASHER }
enum OrderStatus {
    PENDING; WAITING_QUEUE; IN_PROGRESS
    READY; COMPLETED; CANCELED
}
enum VehicleType { CAR; MOTORCYCLE; TRUCK; VAN; SUV }

class Tenant {
    +id; +name; +slug; +isActive: Boolean
    +saasPlan; +monthlyGoal: Float
    +loyaltyInterval: Int; +loyaltyDiscount: Float
}
class User {
    +id; +tenantId?; +name; +email
    +password; +role: UserRole
}
class Customer {
    +id; +tenantId; +name; +phone?
    +email?; +totalWashes: Int
}
class Vehicle {
    +id; +tenantId; +customerId
    +plate; +brand?; +model?; +color?
    +year?; +type: VehicleType
}
class Order {
    +id; +tenantId; +customerId; +vehicleId
    +status: OrderStatus; +total: Float
    +advancePayment: Float; +paymentMethod?
    +startedAt?; +finishedAt?; +completedAt?
    --
    +criarOS(): void
    +atualizarStatus(s: OrderStatus): void
    +calcularTotal(): Float
    +concluir(method: String): void
    +cancelar(): void
}
class OrderItem {
    +id; +orderId; +productId?
    +name; +quantity: Float
    +unitPrice: Float; +isService: Boolean
}
class Product {
    +id; +tenantId; +name; +category?
    +price: Float; +stock: Float
    +minStock: Float; +unit; +isService: Boolean
    --
    +verificarEstoqueMinimo(): Boolean
    +decrementarEstoque(qty: Float): void
}
class Appointment {
    +id; +tenantId; +customerId
    +vehicleId?; +orderId?
    +date: DateTime; +status; +notes?
    --
    +converterEmOS(): Order
    +cancelar(): void
}
class Employee {
    +id; +tenantId; +name; +role
    +salary: Float; +commissionPct: Float
    +isActive: Boolean; +userId?

```

```

--
+calcularComissao(total: Float): Float
}
class FinancialTransaction {
+id; +tenantId; +orderId?
+type; +category; +description
+amount: Float; +status; +paymentMethod?
}
class InventoryTransaction {
+id; +tenantId; +productId; +orderId?
+type; +quantity: Float
}
class FixedExpense {
+id; +tenantId; +name
+amount: Float; +dueDay: Int; +isActive: Boolean
--
+lancarMensalmente(): FinancialTransaction
}

Tenant "1" *-- "0..*" User
Tenant "1" *-- "0..*" Customer
Tenant "1" *-- "0..*" Vehicle
Tenant "1" *-- "0..*" Order
Tenant "1" *-- "0..*" Product
Tenant "1" *-- "0..*" Employee
Tenant "1" *-- "0..*" Appointment
Tenant "1" *-- "0..*" FinancialTransaction
Tenant "1" *-- "0..*" InventoryTransaction
Tenant "1" *-- "0..*" FixedExpense
Customer "1" -- "0..*" Vehicle
Customer "1" -- "0..*" Order
Customer "1" -- "0..*" Appointment
Vehicle "1" -- "0..*" Order
Order "1" *-- "1..*" OrderItem
Order "1" -- "0..1" Appointment
OrderItem "0..*" -- "0..1" Product
Product "1" -- "0..*" InventoryTransaction
Employee "0..1" -- "0..1" User
@enduml

```

Fonte: Elaborado pelo autor (2026).

4.3 Diagrama de Objetos

O diagrama de objetos apresenta um *snapshot* concreto do sistema, representando o cenário em que o cliente **João Silva** deixou seu veículo **Honda Civic (ABC-1234)** para lavagem completa com cera, com a OS em execução (*IN_PROGRESS*).

As classes **Order**, **Customer**, **Vehicle**, **OrderItem** e **Product** foram selecionadas por constituírem o núcleo transacional do sistema — são as entidades que trocam estado com maior frequência durante a operação diária. **Employee** e **FinancialTransaction** foram omitidos pois ainda não possuem valores definitivos enquanto a OS está em andamento.

Figura 4 — Diagrama de Objetos: OS em execução

```

@startuml DiagramaObjetos_WashControl

object "lavaJatoSul : Tenant" as t {
    id          = "clx001tenant"
    name        = "Lava Jato Sul"
    slug        = "lava-jato-sul"
    saasPlan    = "PRO"
    loyaltyInterval = 10
}
object "joao : Customer" as c {
    id          = "clx002customer"
    tenantId    = "clx001tenant"
    name        = "João Silva"
    phone       = "(41) 99999-1234"
    totalWashes = 7
}
object "civic : Vehicle" as v {
    id          = "clx003vehicle"
    tenantId    = "clx001tenant"
    plate       = "ABC-1234"
    brand       = "Honda"
    model       = "Civic"
    year        = 2021
    type        = CAR
}
object "os001 : Order" as o {
    id          = "clx004order"
    tenantId    = "clx001tenant"
    status      = IN_PROGRESS
    total       = 120.00
    startedAt   = "2026-04-17 09:30"
    completedAt = null
}
object "item1 : OrderItem" as i1 {
    name        = "Lavagem Completa"
    quantity    = 1.0
    unitPrice   = 80.00
    isService   = true
}
object "item2 : OrderItem" as i2 {
    name        = "Cera Automotiva"
    quantity    = 1.0
    unitPrice   = 40.00
    isService   = false
}
object "ceraCarnauba : Product" as p {
    name        = "Cera Automotiva Carnaúba 500g"
    stock       = 12.0
    minStock    = 3.0
    isService   = false
}

t --> c ; t --> v ; t --> o
c --> v ; c --> o ; v --> o
o --> i1 ; o --> i2 ; i2 --> p
@enduml

```

Fonte: Elaborado pelo autor (2026).

5 RA3 — DIAGRAMAS COMPORTAMENTAIS

5.1 Diagramas de Sequência

Os cinco diagramas de sequência a seguir modelam as principais interações do WashControl, utilizando fragmentos combinados (alt, loop, opt) para representar condições, repetições e comportamentos opcionais.

DS-01 — Autenticação e Acesso ao Dashboard

Figura 5 — DS-01: Autenticação

```
@startuml DS01_Autenticacao
actor Manager
participant "LoginForm (Client)" as UI
participant "Middleware (auth)" as MID
participant "NextAuth Action" as AUTH
database "PostgreSQL" as DB

Manager -> UI : informa e-mail e senha
UI -> AUTH : signIn(email, password)
AUTH -> DB : SELECT User WHERE email = ?

alt Usuário encontrado
  DB --> AUTH : { id, name, role, tenantId, hash }
  AUTH -> AUTH : bcrypt.compare(password, hash)
  alt Senha correta
    AUTH --> UI : JWT { id, role, tenantId }
    UI -> MID : GET /dashboard
    MID -> MID : verifica JWT, extrai role/tenantId
    MID --> UI : acesso permitido
    UI --> Manager : exibe Dashboard com KPIs
  else Senha incorreta
    AUTH --> UI : erro: "Credenciais inválidas"
    UI --> Manager : exibe mensagem de erro
  end
end
else Usuário não encontrado
  DB --> AUTH : null
  AUTH --> UI : erro: "Usuário não encontrado"
  UI --> Manager : exibe mensagem de erro
end
@enduml
```

Fonte: Elaborado pelo autor (2026).

DS-02 — Criar e Concluir Ordem de Serviço

Figura 6 — DS-02: Ciclo completo da OS

```

@startuml DS02_CriarConcluirOS
actor Manager
actor Washer
participant "Dashboard (Client)" as UI
participant "Server Action (OS)" as SA
participant "Prisma ORM" as ORM
database "PostgreSQL" as DB

== Criação ==
Manager -> UI : preenche formulário (cliente, veículo, itens)
UI -> SA : criarOS(tenantId, customerId, vehicleId, items[])
SA -> ORM : order.create({ status: PENDING, items })
ORM -> DB : INSERT Order + OrderItem(s)
DB --> ORM : orderId
SA --> UI : revalidatePath
UI --> Manager : OS no Kanban - coluna PENDING

== Fila ==
Manager -> UI : move para WAITING_QUEUE
UI -> SA : atualizarStatus(orderId, WAITING_QUEUE)
SA -> ORM : order.update({ status: WAITING_QUEUE })
ORM -> DB : UPDATE Order
DB --> ORM : OK
SA --> UI : OK

== Lavador inicia ==
Washer -> UI : clica "Iniciar"
UI -> SA : atualizarStatus(orderId, IN_PROGRESS)
SA -> ORM : order.update({ status: IN_PROGRESS, startedAt: now() })
ORM -> DB : UPDATE Order
DB --> ORM : OK
SA --> UI : OK

== Lavador finaliza ==
Washer -> UI : clica "Finalizado"
UI -> SA : atualizarStatus(orderId, READY)
SA -> ORM : order.update({ status: READY, finishedAt: now() })
ORM -> DB : UPDATE Order
DB --> ORM : OK
SA --> UI : OK

== Conclusão com pagamento ==
Manager -> UI : confirma pagamento + método
UI -> SA : concluirOS(orderId, paymentMethod, total)
SA -> ORM : order.update({ status: COMPLETED, completedAt: now() })
ORM -> DB : UPDATE Order
SA -> ORM : financialTransaction.create(INCOME, total)
ORM -> DB : INSERT FinancialTransaction
loop cada OrderItem com isService = false
  SA -> ORM : inventoryTransaction.create(OUT, qty)
  ORM -> DB : INSERT InventoryTransaction
  SA -> ORM : product.update(stock - qty)
  ORM -> DB : UPDATE Product.stock
end
SA -> ORM : customer.update(totalWashes + 1)
ORM -> DB : UPDATE Customer
DB --> ORM : OK
SA --> UI : sucesso
UI --> Manager : OS -> COMPLETED
@enduml

```

Fonte: Elaborado pelo autor (2026).

DS-03 — Criar Agendamento e Converter em OS

Figura 7 — DS-03: Agendamento

```
@startuml DS03_Agendamento
actor Manager
participant "Dashboard (Client)" as UI
participant "Server Action (Agenda)" as SA
participant "Prisma ORM" as ORM
database "PostgreSQL" as DB

== Criação do Agendamento ==
Manager -> UI : preenche data, cliente, veículo
UI -> SA : criarAgendamento(tenantId, dados)
SA -> ORM : appointment.findMany(date range)
ORM -> DB : SELECT conflitos de horário
DB --> ORM : [] (disponível)
SA -> ORM : appointment.create({ status: SCHEDULED })
ORM -> DB : INSERT Appointment
SA --> UI : sucesso
UI --> Manager : agendamento no calendário

== Conversão em OS ==
Manager -> UI : clica "Converter em OS"
UI -> SA : converterEmOS(appointmentId)
SA -> ORM : appointment.findUnique(id)
ORM -> DB : SELECT Appointment
DB --> ORM : { orderId: null }

alt orderId == null
  SA -> ORM : order.create({ status: PENDING })
  ORM -> DB : INSERT Order
  SA -> ORM : appointment.update({ orderId, status: COMPLETED })
  ORM -> DB : UPDATE Appointment
  SA --> UI : redireciona para Kanban
else orderId != null
  SA --> UI : erro: "OS já vinculada"
  UI --> Manager : alerta com link para OS existente
end
enduml
```

Fonte: Elaborado pelo autor (2026).

DS-04 — Baixa Automática de Estoque ao Concluir OS

Figura 8 — DS-04: Baixa de estoque

```

@startuml DS04_BaixaEstoque
actor Manager
participant "Dashboard (Client)" as UI
participant "Server Action" as SA
participant "Prisma ORM" as ORM
database "PostgreSQL" as DB

Manager -> UI : confirma conclusão da OS
UI -> SA : concluirOS(orderId, paymentMethod)
SA -> ORM : orderItem.findMany({ orderId, isService: false })
ORM -> DB : SELECT insumos da OS
DB --> ORM : [ {productId, quantity}, ... ]

loop para cada insumo
  SA -> ORM : product.findUnique(productId)
  ORM -> DB : SELECT Product
  DB --> ORM : { stock, minStock }

  alt stock >= quantity
    SA -> ORM : inventoryTransaction.create(OUT)
    ORM -> DB : INSERT InventoryTransaction
    SA -> ORM : product.update(stock - quantity)
    ORM -> DB : UPDATE Product
    opt novo stock < minStock
      SA --> UI : alerta estoque mínimo atingido
    end
  else stock insuficiente
    SA --> UI : aviso: estoque negativo registrado
    SA -> ORM : product.update(stock - quantity)
    ORM -> DB : UPDATE Product (valor negativo)
  end
end
SA --> UI : baixas concluídas
UI --> Manager : OS → COMPLETED
@enduml

```

Fonte: Elaborado pelo autor (2026).

DS-05 — Visualização do Relatório Financeiro (DRE)

Figura 9 — DS-05: DRE Financeiro

```

@startuml DS05_Financeiro
actor Manager
participant "Page (Server Component)" as SC
participant "Server Action (Finance)" as SA
participant "Prisma ORM" as ORM
database "PostgreSQL" as DB

Manager -> SC : GET /dashboard/financeiro?mes=4&ano=2026
SC -> SA : getDadosFinanceiros(tenantId, mes, ano)
SA -> ORM : financialTransaction.findMany(INCOME, range)
ORM -> DB : SELECT receitas do periodo
DB --> ORM : FinancialTransaction[]
SA -> ORM : financialTransaction.findMany(EXPENSE, range)
ORM -> DB : SELECT despesas do periodo
DB --> ORM : FinancialTransaction[]
SA -> ORM : fixedExpense.findMany({ isActive: true })
ORM -> DB : SELECT despesas fixas ativas
DB --> ORM : FixedExpense[]
SA -> SA : calcularDRE() -> { receitas, despesas, lucro, margem% }
SA --> SC : { kpis, transacoes[], dre }
SC --> Manager : Dashboard com KPIs, gráfico e DRE

opt Manager altera período
  Manager -> SA : getDadosFinanceiros(novoMes, novoAno)
  note right : repete fluxo acima
end
@enduml

```

Fonte: Elaborado pelo autor (2026).

5.2 Diagramas de Atividades

Os quatro diagramas de atividades a seguir modelam os principais processos do WashControl, utilizando construções de paralelismo (*fork/join*) e condições de desvio (*if/else*), conforme exigido pela notação UML.

DA-01 — Fluxo Completo de Atendimento (Ciclo da OS)

Figura 10 — DA-01: Ciclo completo da OS com raias por ator

```

@startuml DA01_FluxoOS
    |Manager|
    start
    :Recebe veículo e busca cliente no sistema;
    :Seleciona veículo e adiciona itens à OS;
    :Confirma criação (status: PENDING);
    |Sistema|
    :Persiste OS; exibe no Kanban - PENDING;
    |Manager|
    :Move OS para WAITING_QUEUE;
    |Washer|
    :Visualiza fila e seleciona OS; clica "Iniciar";
    |Sistema|
    :Status → IN_PROGRESS; startedAt = now();
    |Washer|
    :Executa serviços; clica "Finalizado";
    |Sistema|
    :Status → READY; finishedAt = now();
    |Manager|
    :Recebe pagamento;
    fork
        :Registra método de pagamento;
    fork again
        |Sistema|
        :Dispara notificação ao cliente (RF09);
    end fork
    |Manager|
    :Confirma conclusão da OS;
    |Sistema|
    :Status → COMPLETED; completedAt = now();
    fork
        :Cria FinancialTransaction (INCOME);
    fork again
        :Baixa estoque dos insumos (OUT);
    fork again
        :Customer.totalWashes++;
    end fork
    if (totalWashes % loyaltyInterval == 0?) then (Sim)
        :Aplica/sinaliza desconto de fidelidade;
    else (Não)
    endif
    :Calcula comissão do funcionário;
    stop
@enduml

```

Fonte: Elaborado pelo autor (2026).

DA-02 — Fluxo de Agendamento

Figura 11 — DA-02: Agendamento com idempotência

```
@startuml DA02_Agendamento
start
:Manager cria agendamento (data, cliente, veículo);
|Sistema|
:Verifica disponibilidade do horário;
if (Horário disponível?) then (Sim)
  :Salva Appointment - SCHEDULED;
  fork
    :Exibe no calendário;
  fork again
    :Envia lembrete ao cliente (RF09 - futuro);
  end fork
  :Data do atendimento chega;
  if (Cliente comparece?) then (Sim)
    |Manager|
    :Clica "Converter em OS";
    |Sistema|
    if (orderId == null?) then (Sim)
      :Cria Order PENDING; vincula orderId;
      :Appointment -> COMPLETED;
    else (Não - OS já existe)
      :Bloqueia criação duplicada;
      :Exibe link para OS existente;
    endif
  endif
  else (Não comparece)
    |Manager|
    :Cancela agendamento;
    |Sistema|
    :Appointment -> CANCELED;
  endif
  else (Horário ocupado)
    :Exibe aviso de conflito;
    :Retorna ao formulário;
  endif
endif
stop
@enduml
```

Fonte: Elaborado pelo autor (2026).

DA-03 — Cálculo de Fidelidade e Desconto

Figura 12 — DA-03: Programa de fidelidade

```

@startuml DA03_Fidelidade
start
:OS concluída (status → COMPLETED);
|Sistema|
:Customer.totalWashes + 1;
:Consulta: loyaltyInterval e loyaltyDiscount do Tenant;
if (totalWashes % loyaltyInterval == 0?) then (Sim)
  :Cliente atingiu marco de fidelidade;
  fork
    :Registra desconto para próxima OS;
  fork again
    :Notifica cliente sobre bônus (RF09 - futuro);
  end fork
if (loyaltyDiscount == 100?) then (Lavagem grátis)
  :Próxima OS com valor = R$ 0,00;
else (Desconto parcial)
  :Próxima OS com {loyaltyDiscount}% de desconto;
endif
else (Não)
  :Exibe progresso: "X de N lavagens";
endif
stop
@enduml

```

Fonte: Elaborado pelo autor (2026).

DA-04 — Lançamento de Despesa Fixa Mensal

Figura 13 — DA-04: Despesa fixa recorrente

```

@startuml DA04_DespesaFixa
start
:Sistema verifica data atual;
:Consulta FixedExpenses onde dueDay == dia_atual;
if (Existem despesas a lançar?) then (Sim)
  repeat
    :Seleciona próxima FixedExpense;
    :Verifica idempotência (já lançada este mês?);
    if (Já lançada?) then (Sim)
      :Ignora - evita duplicidade;
    else (Não)
      :Cria FinancialTransaction (EXPENSE);
      :Persiste no banco;
    endif
  repeat while (Há mais despesas?)
    :Notifica Manager sobre despesas lançadas;
  else (Nenhuma despesa hoje)
    :Nenhuma ação;
  endif
stop
@enduml

```

Fonte: Elaborado pelo autor (2026).

5.3 Diagramas de Máquina de Estados

Foram identificados dois elementos do sistema com ciclo de vida relevante para modelagem: a **Ordem de Serviço** e o **Agendamento**. Ambos os diagramas contemplam ações de entrada (*entry*), ações de saída (*exit*), condições de guarda e eventos, conforme exigido pela notação UML.

DME-01 — Ciclo de Vida da Ordem de Serviço

Figura 14 — DME-01: Estados da Ordem de Serviço

```
@startuml DME01_EstadosOS
[*] --> PENDING : criarOS() [cliente + veículo válidos]

state PENDING {
    entry: registrar createdAt = now()
    exit:  validar mínimo 1 item na OS
}
PENDING --> WAITING_QUEUE : confirmarEntrada() [OS possui >= 1 item]
PENDING --> CANCELED      : cancelar() [apenas MANAGER]

state WAITING_QUEUE {
    entry: notificar pátio sobre novo veículo
    exit:  registrar posição na fila
}
WAITING_QUEUE --> IN_PROGRESS : iniciarAtendimento() [WASHER ou MANAGER]
WAITING_QUEUE --> CANCELED    : cancelar() [apenas MANAGER]

state IN_PROGRESS {
    entry: startedAt = now()
    exit:  finishedAt = now()
}
IN_PROGRESS --> READY      : concluirServico() [WASHER ou MANAGER]
IN_PROGRESS --> CANCELED   : cancelar() [apenas MANAGER]

state READY {
    entry: notificar cliente / RF09
    exit:  validar método de pagamento
}
READY --> COMPLETED : registrarPagamento(method) [pagamento confirmado]
READY --> IN_PROGRESS : solicitarRetrabalho() [apenas MANAGER]

state COMPLETED {
    entry: completedAt = now()
    entry: criarFinancialTransaction(INCOME)
    entry: baixarEstoque(itens OUT)
    entry: Customer.totalWashes++
    entry: calcularComissaoFuncionario()
}
COMPLETED --> [*]
CANCELED --> [*]
@enduml
```

Fonte: Elaborado pelo autor (2026).

DME-02 — Ciclo de Vida do Agendamento

Figura 15 — DME-02: Estados do Agendamento

```

@startuml DME02_EstadosAppointment
[*] --> SCHEDULED : criarAgendamento() [horário disponível]

state SCHEDULED {
    entry: salvar data e hora
    entry: enviar confirmação ao cliente (RF09)
    exit: verificar orderId == null (idempotência)
}

SCHEDULED --> COMPLETED : converterEmOS() [cliente compareceu]
    note on link : / vincular orderId à OS criada
SCHEDULED --> CANCELED : cancelar() [Manager ou Cliente]
SCHEDULED --> CANCELED : naoComparecimento() [Manager registra ausência]

state COMPLETED {
    entry: orderId = novaOS.id
    entry: liberar slot para nova marcação
}

state CANCELED {
    entry: liberar slot de horário na agenda
    entry: notificar cliente (RF09)
}

COMPLETED --> [*]
CANCELED --> [*]

note right of SCHEDULED
    Guarda de idempotência:
    [orderId == null] → pode converter em OS
    [orderId != null] → bloqueado (OS já existe)
end note
@enduml

```

Fonte: Elaborado pelo autor (2026).

6 PROTÓTIPO E NAVEGAÇÃO DE TELAS

O WashControl foi desenvolvido como um protótipo de **alta fidelidade funcional**, implementado em Next.js 15 com banco de dados real (PostgreSQL), autenticação e Server Actions. Todas as telas são interativas e operacionais, superando o escopo de um protótipo estático.

6.1 Diagrama de Navegação

Figura 16 — Diagrama de navegação de telas

```
@startuml NavegacaoTelas
[*] --> Login

Login --> Dashboard      : autenticação OK

Dashboard --> OS          : Ordens de Serviço
Dashboard --> Patio      : Pátio (Washer)
Dashboard --> Clientes    : Clientes e Veículos
Dashboard --> Agendamentos
Dashboard --> Financeiro
Dashboard --> Insumos
Dashboard --> Equipe
Dashboard --> Lavacarros  : Catálogo de Serviços
Dashboard --> Configuracoes
Dashboard --> Faturamento : (SuperAdmin)

OS --> Agendamentos      : converter agendamento
Agendamentos --> OS      : OS criada
OS --> Financeiro        : OS concluída → lança receita
OS --> Insumos           : OS concluída → baixa estoque
@enduml
```

Fonte: Elaborado pelo autor (2026).

6.2 Telas Principais do Sistema

Quadro 3 — Telas do WashControl e seus objetivos

Tela / Rota	Descrição e Funcionalidades	Ator Principal
/login	Autenticação com e-mail e senha. Redireciona automaticamente para o dashboard conforme o papel do usuário (MANAGER → gestão completa; WASHER → pátio).	Todos os atores
/dashboard	KPIs em tempo real: receita do dia, receita do mês, OSs em andamento e meta mensal. Gráfico comparativo de receitas × despesas. Lista de OSs recentes.	Manager
/dashboard/os	Board Kanban com cinco colunas de status. Drag-and-drop de cards. Modal de criação de OS e modal de conclusão com registro de pagamento.	Manager
/dashboard/patio	Visão simplificada e responsiva dos veículos em atendimento. Botões "Iniciar" e "Finalizado" acessíveis via celular no pátio.	Washer
/dashboard/clientes	CRUD de clientes com veículos vinculados, histórico de lavagens e indicador do programa de fidelidade.	Manager
/dashboard/agendamentos	Calendário mensal de agendamentos, formulário de novo agendamento e botão de conversão em OS com verificação de idempotência.	Manager
/dashboard/financeiro	DRE mensal, fluxo de caixa, listagem de transações filtráveis e lançamento manual de despesas.	Manager
/dashboard/insumos	Catálogo de produtos com controle de estoque, alertas de estoque mínimo e histórico de movimentações.	Manager
/dashboard/equipe	Cadastro de funcionários, vinculação com usuário do sistema e relatório de comissões por OS concluída.	Manager
/dashboard/configuracoes	Dados do tenant, configuração do programa de fidelidade (loyaltyInterval e	Manager

Tela / Rota	Descrição e Funcionalidades	Ator Principal
	loyaltyDiscount) e plano SaaS atual.	

Fonte: Elaborado pelo autor (2026).

7 CRIATIVIDADE

Os itens abaixo foram identificados pela equipe como contribuições criativas — itens não solicitados explicitamente pelo enunciado que tornam o trabalho superior em relação ao mínimo esperado. O item de criatividade pode relacionar-se ao projeto (documento) e ao produto que está sendo gerado.

7.1 Criatividade no Projeto (Documento)

C01 — Protótipo de alta fidelidade funcional

Em vez de um protótipo estático no Figma, o WashControl foi desenvolvido como aplicação funcional em Next.js 15 com banco de dados real (PostgreSQL), autenticação e Server Actions — tornando o protótipo interativo, demonstrável e com dados reais de teste via seed.

C02 — Técnicas de Elicitação com rastreabilidade completa

Cada técnica utilizada (Entrevista, Observação, Questionário, Brainstorming, JAD) foi documentada com stakeholder-alvo, roteiro aplicado, limitações encontradas e mapeamento direto para os RFs/RNFs resultantes — indo além da simples listagem de técnicas exigida.

C03 — Diagrama de Objetos com cenário real e justificativa

O diagrama de objetos apresenta uma instância concreta de atendimento em andamento, com valores reais e justificativa explícita das classes representadas e das omitidas — evidenciando compreensão profunda da diferença entre diagrama de classes e diagrama de objetos.

C04 — Idempotência modelada como regra de negócio explícita

A restrição `Appointment.orderId @unique` foi especificada como regra de negócio (RN-03) e representada em fluxos alternativos, fluxos de exceção e no diagrama de

estados do Agendamento — demonstrando maturidade na modelagem de restrições de integridade.

7.2 Criatividade no Produto

C05 — Arquitetura multi-tenant com isolamento estrutural por tenantId

O isolamento de dados entre estabelecimentos é garantido em nível de banco de dados: o campo `tenantId` é obrigatório em todas as tabelas, com índice composto para eficiência — não é um isolamento lógico por aplicação, mas arquitetural e auditável.

C06 — Automações em cascata ao concluir OS (5 ações em transação única)

Ao marcar uma OS como `COMPLETED`, cinco ações são executadas atômicamente: (1) receita financeira, (2) baixa de estoque por insumo, (3) contagem de fidelidade, (4) verificação de desconto e (5) cálculo de comissão do funcionário — modelado e implementado.

C07 — Programa de fidelidade parametrizável por tenant

Cada lava-car configura seu próprio programa de fidelidade via `loyaltyInterval` (a cada N lavagens) e `loyaltyDiscount` (X% de desconto) no modelo Tenant — um diferencial competitivo que vai além dos requisitos documentados.

C08 — Módulo de faturamento SaaS B2B para o Super Admin

Além do sistema para o lava-car, existe um módulo de faturamento onde o Super Admin gerencia planos (BASIC, PRO, ENTERPRISE), valores de assinatura e datas de vencimento dos tenants — tornando o WashControl um produto SaaS completo, não apenas um sistema de gestão isolado.

8 RESPONSABILIDADES

Todos os integrantes da equipe devem assinar e datar o documento de Análise e Projeto do Sistema Computacional, declarando ciência e responsabilidade pelo conteúdo apresentado.

Integrante	RA / Matrícula	Assinatura	Data
Leonardo dos Santos Marques			17/04/2026

Documento elaborado para a disciplina de **Modelagem de Sistemas Computacionais** —
BCC/PUCPR

Professora: Patricia Rucker de Bassi | Semestre: 2026-1
Sistema: **WashControl** — SaaS de Gestão para Lava-Cars